

A Lightweight, Privacy-Preserving Offline Phishing Detection System Using Explainable URL-Based Analysis

1st Ahmad AlMaghaireh
CyberSecurity Department
Al Hussein Technical University
Amman, Jordan
23110205@htu.edu.jo

2nd Rawaah Al-Qoraan
CyberSecurity Department
Al Hussein Technical University
Amman, Jordan
Rawaah.Qoraan@htu.edu.jo

Abstract— Phishing attacks remain a persistent cybersecurity threat, exploiting deceptive web content and misleading URLs to compromise user credentials and sensitive information. While machine-learning-based phishing detection has shown strong predictive performance, many existing approaches rely on cloud-based inference, external reputation services, or intrusive warning mechanisms, which introduce latency, privacy concerns, and usability challenges. This paper presents a fully offline, privacy-preserving phishing URL detection system designed for real-time, browser-native deployment. The proposed approach relies exclusively on intrinsic URL-based lexical and structural features, enabling detection without network access or webpage content retrieval. A lightweight XGBoost classifier is employed to balance high detection accuracy with low inference latency, making the system suitable for synchronous execution during browsing. To enhance transparency and user trust, SHapley Additive exPlanations (SHAP) are integrated to provide interpretable, feature-level explanations for individual decisions. Rather than interrupting user interaction, warnings are presented through a non-intrusive inline visual cue, reducing alert fatigue while maintaining security awareness. The system is evaluated using the PhiUSIIL phishing URL dataset, which reflects contemporary phishing characteristics. Experimental results show that the URL-only model achieves 99.9% accuracy, with both phishing miss rates and legitimate false-alarm rates remaining below 0.1%, while maintaining millisecond-level inference latency. A prototype browser extension further validates deployability and explainability under strict privacy constraints. These results indicate that accurate, explainable, and privacy-conscious phishing detection can be achieved entirely on-device, providing a practical foundation for modern browser security systems.

Keywords— *Phishing detection, URL-based analysis, Explainable artificial intelligence, Privacy-preserving security, Browser-based protection*

I. INTRODUCTION

The rapid expansion of internet technologies and the widespread adoption of online platforms have significantly increased society's reliance on digital systems for communication, financial transactions, and business operations. As digital ecosystems continue to grow in scale and complexity, they increasingly attract malicious actors who exploit system vulnerabilities for financial gain or data theft [1]. Consequently, cybersecurity has become a critical discipline concerned with protecting systems, networks, and users from unauthorized access, misuse, and disruption. Despite advances in defensive technologies, modern web environments—characterized by dynamic content, distributed services, and large-scale user interaction—remain particularly susceptible to sophisticated cyber threats,

resulting in substantial economic and reputational losses worldwide [1].

Among these threats, phishing attacks represent one of the most pervasive and effective forms of social engineering. Phishing exploits human trust by impersonating legitimate services and directing users to fraudulent websites designed to harvest sensitive information, such as login credentials or financial data [2]. Attackers frequently rely on deceptive Uniform Resource Locators (URLs) that visually resemble trusted domains or embed misleading structural patterns. Traditional phishing defenses, including blacklist- and reputation-based systems, are limited by delayed reporting, periodic updates, and dependence on external services, allowing many short-lived phishing campaigns to evade detection. As a result, research has increasingly shifted toward machine-learning-based phishing detection, particularly approaches that analyze intrinsic URL characteristics and enable rapid, click-time classification without remote queries [2].

While machine-learning models have demonstrated promising detection accuracy, many existing solutions rely on cloud-based inference, external feature lookups, or content-level analysis, which introduce several practical limitations. These include increased inference latency, potential leakage of user browsing behavior to third-party servers, and reliance on intrusive warning mechanisms such as blocking pages or disruptive pop-up alerts. Such constraints limit the suitability of these approaches for privacy-sensitive, real-time deployment in modern browsers. Addressing these challenges requires phishing detection systems that are not only accurate, but also lightweight, privacy-preserving, and seamlessly integrable into the browsing experience.

In this work, we investigate whether a fully offline, URL-only phishing detection model can achieve high predictive performance while remaining suitable for real-time, browser-native deployment under strict privacy and latency constraints. In addition, we examine whether combining lightweight detection with inline, explainable warnings can improve transparency and usability without disrupting the browsing experience. To this end, we propose a client-side phishing detection system that integrates intrinsic URL-based features, a lightweight XGBoost classifier, and SHAP-based explanations, and we evaluate its effectiveness using a large, contemporary phishing dataset.

The proposed system is designed to eliminate dependence on external reputation services, network connectivity, and

heavyweight content inspection. By relying exclusively on intrinsic URL features and local inference, the approach preserves user privacy while enabling synchronous phishing detection during navigation events. A prototype browser extension is developed to demonstrate deployment feasibility and to validate that accurate and explainable phishing detection can be achieved entirely on-device.

The rest of this document is organized as follows. Section II presents the literature studies. Section III presents the methodology of this study. Section IV presents the results and discussion. Section V presents the conclusion of this research.

II. LITERATURE REVIEW

Research on phishing detection has expanded rapidly in recent years due to the growing sophistication of social engineering attacks and the need for fast, reliable defenses. Prior studies span user-centric interaction mechanisms, deep-learning-based URL classification, explainable artificial intelligence (XAI), and privacy-preserving client-side deployment. The following works are reviewed to position recent advances and identify remaining limitations addressed by this study.

Lain et al. [3] examined whether active user engagement could improve phishing detection compared to passive browser warnings. They proposed three interaction-based URL inspection tasks—clicking, highlighting, and typing the legitimate domain—and evaluated them through a cross-country user study involving 2,673 participants. Results showed a significant reduction in phishing susceptibility, with click-through rates dropping from 74% to 35%. Despite their effectiveness, such approaches impose cognitive overhead that may limit scalability in routine browsing.

Roy et al. [4] introduced PhishXplain, a real-time explainable phishing warning framework that generates contextual explanations using a lightweight on-device LLaMA-3.2 3B language model. Evaluated on 7,091 live phishing websites, the system achieved 96% precision with an average response time of 5.2 seconds and demonstrated improved user comprehension in a controlled study. However, reliance on large language models introduces computational overhead that challenges seamless browser-native deployment.

Ahubele et al. [5] proposed an interpretable phishing website detection system combining Random Forest and XGBoost classifiers with SHAP explanations. Using lexical, heuristic, and semantic features, the system achieved 95.0% accuracy and 92.0% recall, and was deployed as a browser extension with a backend service. While interpretable, the dependency on server-side infrastructure increases latency and deployment complexity.

Uddin et al. [6] explored explainable phishing detection using XGBoost and SHAP on a Kaggle dataset, reporting 97–98% accuracy and consistent feature-level interpretability. Their work highlights the effectiveness of tree-based models with SHAP, but focuses primarily on

offline evaluation and does not address real-time, client-side constraints.

ul Haq et al. [7] proposed a character-level deep-learning approach using a one-dimensional CNN trained on approximately 400,000 URLs. The model achieved 99.3% accuracy and strong robustness against obfuscation techniques; however, the computational cost of convolutional architectures limits suitability for continuous browser execution.

Baskota et al. [8] addressed phishing detection as a multi-class problem using a Bi-LSTM model on a large Kaggle dataset. Although overall accuracy reached 98%, phishing-specific performance was lower, indicating the difficulty of isolating phishing behavior in multi-class settings.

Almohameed et al. [9] presented a hybrid CNN–BiGRU architecture enhanced with SMOTE to address class imbalance. Evaluated on the Kaggle Malicious URLs dataset, the model achieved 98.46% accuracy and high AUC, but its architectural complexity limits low-latency deployment.

Acharya et al. [10] proposed a fully client-side phishing detection browser extension using a Random Forest–XGBoost ensemble trained on URL, host, and HTML features. The system achieved high accuracy across phishing URLs, emails, and websites with inference latency below 150 ms, but did not incorporate explainable decision outputs.

Wang et al. [11] introduced WebSHAP, a framework enabling SHAP explanations to be computed entirely within web browsers using WebAssembly and WebGL. Although not phishing-specific, this work provides an important foundation for privacy-preserving, client-side explainability.

Sharma et al. [12] demonstrated that ensemble learning can be adapted to resource-constrained environments through a TinyML stacking approach, achieving 99.98% accuracy on the ToN-IoT dataset with minimal latency. While focused on IoT intrusion detection, the study highlights the feasibility of lightweight, high-performance models under strict computational constraints.

Despite substantial progress, important gaps remain. Interaction-based defenses impose cognitive overhead, explainable systems relying on large language models or backend services introduce latency and privacy concerns, and deep-learning-based URL classifiers are often too computationally intensive for browser-native use. Moreover, many studies emphasize accuracy without consistently addressing recall, false negatives, or deployment feasibility. Consequently, there is limited evidence of solutions that simultaneously provide fully local execution, lightweight yet expressive models, and non-intrusive, real-time explainability for browser-based phishing detection—an intersection this work seeks to address.

III. METHODOLOGY

This section describes the experimental methodology adopted to design, train, and evaluate the proposed phishing detection system. It outlines the dataset selection, preprocessing strategy, model design and interpretability framework, and evaluation metrics, with particular emphasis on offline execution, privacy preservation, and real-time browser deployment constraints. The overall workflow—from feature selection to model inference and evaluation—is illustrated in Fig. 1.

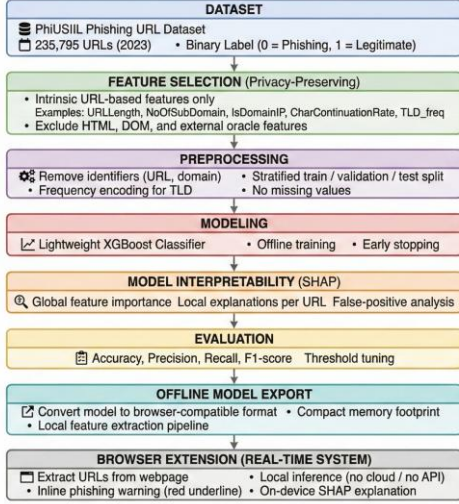


Fig. 1. End-to-end methodology for URL-only phishing detection with on-device inference and explainability.

A. Dataset

This study utilizes the PhiUSIIL dataset [13], a large-scale phishing detection corpus containing over 235,000 URLs collected in 2023. Each instance is annotated with a binary target variable named Label, where phishing URLs are assigned the value 0 and legitimate URLs the value 1, enabling supervised learning and evaluation under contemporary web conditions.

The dataset provides a diverse set of URL- and webpage-related attributes. A major component consists of intrinsic URL-based lexical and structural features extracted directly from the URL string. Examples include URLLength, NoOfSubDomain, IsDomainIP, CharContinuationRate, and TLD_freq, which capture common obfuscation and impersonation patterns exploited in phishing attacks. These features are lightweight to compute and do not require network access, making them well suited for real-time detection.

In addition, the dataset includes several binary indicator features encoding high-level phishing cues, such as HTTPS, HasPasswordField, and the presence of financial or social-network-related terms. While the dataset also contains HTML- and DOM-based count features (e.g., script, iframe, and redirect counts) and external or oracle-like attributes (e.g., similarity and reputation metrics), these features depend on webpage retrieval or external references and introduce additional latency.

To support a privacy-preserving, offline-first deployment model, this study restricts training to a compact subset of

intrinsic URL-based and lightweight structural features, excluding HTML-based and external attributes. The PhiUSIIL dataset was selected due to its scale, diversity, and recent collection date, which improves robustness against concept drift and enhances relevance to modern phishing campaigns. Domain-aware evaluation strategies are applied to mitigate residual dataset bias.

B. Preprocessing

Preprocessing transforms the raw PhiUSIIL dataset into a form suitable for training lightweight, offline machine learning models. The dataset contains 235,795 URL instances with engineered features and a binary target variable (Label), where phishing and legitimate URLs are encoded as 0 and 1, respectively.

To satisfy privacy-preserving and client-side deployment constraints, all features that depend on external knowledge sources or global statistics are excluded prior to model training. This includes oracle-like attributes such as URL similarity metrics, probabilistic top-level domain legitimacy estimates, and title–URL matching scores, which implicitly rely on external corpora or historical data. In addition, non-predictive identifier fields, including the raw URL string and domain name, are removed from direct model input. The remaining feature set consists exclusively of intrinsic, oracle-free URL attributes, including lexical characteristics (e.g., URLLength, NoOfSubDomain, CharContinuationRate), lightweight structural indicators (e.g., IsDomainIP), and binary flags capturing common phishing cues. As the dataset contains no missing values, imputation is not required, and feature scaling is omitted due to the tree-based nature of the selected model.

The dataset is partitioned into training, validation, and testing subsets using stratified sampling to preserve the original class distribution. To retain domain-related information without external dependencies, the categorical top-level domain attribute is transformed using frequency-based encoding derived solely from the training data, ensuring compatibility with fully offline inference.

C. Models

The modelling stage focuses on selecting a classification model that satisfies the requirements of offline execution, low inference latency, and robust performance on heterogeneous tabular features. Rather than conducting extensive model comparisons, this study adopts a design-driven approach, selecting a single model that aligns with both the characteristics of the PhiUSIIL dataset and the constraints of browser-native deployment.

Based on these criteria, XGBoost (Extreme Gradient Boosting) is chosen as the classification model. XGBoost is a tree-based ensemble learning method that constructs an additive model of shallow decision trees using gradient boosting. It incorporates regularization mechanisms, shrinkage, and efficient handling of sparse and mixed-type features, making it particularly effective for structured URL-based phishing data [14].

XGBoost is well suited to the intrinsic feature set used in this study, as it captures non-linear interactions among lexical and structural URL indicators without requiring extensive feature engineering. Moreover, its optimized prediction pipeline enables fast inference, which is critical for real-time, client-side phishing detection.

Hyperparameters are selected to balance predictive performance with model compactness, and early stopping based on a validation set is employed to mitigate overfitting. To support practical deployment, the trained XGBoost model is exported into a lightweight, browser-compatible representation and integrated into a prototype browser extension, where all inference is performed locally without external queries. This design ensures fully offline operation and preserves user privacy while enabling instantaneous URL risk assessment during browsing. The trained model is used consistently across all experiments, including performance evaluation, interpretability analysis, and deployment feasibility assessment.

Model Interpretability

Although XGBoost provides strong predictive performance, ensemble-based models are often criticized for limited transparency. In the context of phishing detection—particularly in user-facing security applications—explainability is essential for building trust and understanding false-positive decisions.

To address this limitation, SHapley Additive exPlanations (SHAP) are integrated as a post-hoc interpretability mechanism. SHAP, grounded in cooperative game theory, explains model predictions by attributing them to the marginal contribution of each input feature. This enables both global interpretability, by identifying the most influential features across the dataset, and local interpretability, by explaining individual classification outcomes.

SHAP analysis is applied to the trained XGBoost model to verify that predictions are driven by semantically meaningful phishing indicators, such as excessive URLLength, abnormal NoOfSubDomain values, character-level obfuscation patterns, and rare TLD_freq occurrences. Local explanations are further used to analyze false-positive cases, supporting model debugging and decision-threshold refinement.

By integrating SHAP-based explanations, the proposed system addresses a common limitation of prior phishing detection approaches that prioritize accuracy without transparency. This interpretability layer enhances trustworthiness and supports the deployment of a privacy-preserving phishing detection model suitable for real-world browser environments.

D. Evaluation Metrics

The performance of the proposed model is evaluated using standard binary classification metrics: **accuracy**, **precision**, **recall**, and **F1-score**. These metrics are computed on the held-out test set and are defined using the number of true positives (TP), false positives (FP), true negatives (TN), and false negatives (FN), where the positive class corresponds to phishing URLs.

Accuracy measures the proportion of correctly classified instances and is defined as follows [15]:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \quad (1)$$

Given the approximately balanced class distribution, accuracy provides a meaningful indicator of overall performance.

Precision measures the reliability of phishing predictions by quantifying the fraction of predicted phishing URLs that are truly phishing [15]:

$$\text{Precision} = \frac{TP}{TP + FP} \quad (2)$$

High precision corresponds to a low false-positive rate, which is important to avoid excessive or unnecessary security warnings.

Recall, also known as the true positive rate, captures the proportion of actual phishing URLs correctly identified by the model [15]:

$$\text{Recall} = \frac{TP}{TP + FN} \quad (3)$$

From a security perspective, high recall is critical, as false negatives correspond to undetected phishing attacks.

The F1-score provides a single metric that balances precision and recall by computing their harmonic mean [15]:

$$\text{F1-score} = \frac{2 \cdot (\text{Precision} \cdot \text{Recall})}{\text{Precision} + \text{Recall}} \quad (4)$$

Because phishing detection systems must simultaneously minimize missed attacks and limit false alarms, the F1-score is particularly suitable for comparing classifiers and guiding deployment decisions.

IV. RESULTS AND DISCUSSION

This section presents an evaluation of the proposed offline, oracle-free phishing URL detection system and discusses its implications for real-world browser deployment. Experiments are conducted using the PhiUSIIL dataset and focus on classification performance, threshold sensitivity, latency feasibility, interpretability, and deployment-oriented mitigation mechanisms. Unlike many prior studies that emphasize accuracy alone, this work evaluates performance alongside practical constraints, including privacy preservation, low latency, and user-facing explainability.

Classification Performance

The proposed system is evaluated using accuracy, precision, recall, and F1-score, following the label convention 0 = phishing and 1 = legitimate. Results are reported for the URL-only model configuration in Table 1, which is specifically designed for lightweight, real-time browser deployment.

Table 1. CLASSIFICATION PERFORMANCE OF THE URL-ONLY XGBOOST MODEL

Policy	Thr.	Acc.	Prec.	Rec.	F1	Miss	FA
Balanced	0.63	0.9993	0.9994	0.9993	0.9994	0.0008	0.0007
Low FA	0.30	0.9990	0.9985	0.9997	0.9991	0.0020	0.0003
Low Miss	0.48	0.9992	0.9991	0.9996	0.9993	0.0012	0.0004

Precision (Prec.), recall (Rec.), and F1-score are reported for the legitimate class (Label = 1). “Miss” denotes phishing miss rate, and “FA” denotes legitimate false-alarm rate.

At the balanced operating point, the model achieves 99.93% accuracy, with both phishing miss rate and legitimate

false-alarm rate remaining below 0.1%. Importantly, this level of performance is achieved using intrinsic URL features only, without reliance on external reputation services, DNS queries, webpage content inspection, or network-based signals.

Compared to prior phishing detection approaches that typically report accuracy in the 95–99% range using deep learning architectures or feature-rich pipelines, the proposed system achieves comparable or superior performance while maintaining substantially lower computational and deployment overhead. This demonstrates that carefully selected URL-level features, when paired with a robust tree-based model, are sufficient for high-confidence phishing detection in privacy-constrained environments.

5.2 Threshold Sensitivity and Error Trade-offs

Beyond aggregate metrics, practical deployment requires careful consideration of the trade-off between false positives and missed phishing attacks. A systematic threshold sweep is therefore conducted over the predicted legitimacy probability rather than fixing the decision threshold at 0.5.

The analysis identifies a balanced operating point at $P_{\text{legit}} \approx 0.63$, which minimizes the combined phishing miss rate and legitimate false-alarm rate. In addition, two policy-driven thresholds are evaluated: one prioritizing low false alarms for usability-sensitive browsing contexts and another prioritizing low phishing misses for security-critical environments.

These results show that a single trained model can be flexibly adapted to diverse deployment requirements purely through threshold adjustment, without retraining or architectural modification. This flexibility is particularly valuable for browser-based systems, where tolerance for warnings varies across users and contexts.

5.3 Latency and Browser Deployment Feasibility

Inference latency is a critical constraint for browser-native security tools. Under realistic single-URL inference conditions, feature extraction and model prediction complete within a few milliseconds per URL, remaining well below perceptible delay thresholds for interactive systems.

Because all computation is performed locally and no external queries are issued, latency remains predictable and independent of network conditions. These characteristics confirm that the proposed pipeline is suitable for synchronous execution during navigation events, enabling real-time phishing analysis without disrupting page loading or user interaction.

Inline Detection and Explainable Browser Deployment

To validate real-world applicability, the trained model is integrated into a prototype browser extension that performs inline phishing detection during normal browsing. When a URL is identified as suspicious, the extension applies a non-intrusive inline warning directly within the page context, allowing users to remain aware of potential risk without blocking interaction.

In addition to risk indication, the system presents local SHAP-based explanations associated with the classification decision. These explanations highlight the most influential URL features contributing to the prediction, such as excessive URL length, abnormal subdomain depth, character-level obfuscation patterns, and rare top-level domains. By surfacing

feature-level evidence alongside the warning, the system improves transparency and supports informed user judgment.

Fig. 2 illustrates an example of the inline phishing warning generated by the browser extension together with the corresponding SHAP explanation for the detected URL. This demonstrates that explainable machine-learning decisions can be delivered entirely on-device, without backend services, while remaining compatible with real-time browsing workflows.

Model Interpretability and SHAP Analysis

Beyond the browser interface, SHAP analysis is used to assess model behavior at a global level. Global feature importance indicates that predictions are driven by semantically meaningful phishing indicators, including URL length, character continuation patterns, subdomain count, special-character ratios, and top-level domain frequency. These findings align closely with established phishing heuristics, reinforcing confidence that the model learns plausible decision rules rather than exploiting spurious correlations.

Local SHAP explanations further support error analysis. False positives typically arise from legitimate but structurally atypical URLs, providing actionable insight for threshold tuning and allowlist refinement. This interpretability layer directly addresses the “black-box” criticism commonly associated with gradient-boosted models and enhances trust in user-facing security settings.

Human-in-the-Loop Mitigation via Allowlisting

Despite near-perfect classification performance, residual false positives are unavoidable in real-world security systems. To mitigate this, the proposed system incorporates a deterministic allowlist mechanism applied after model inference.

The allowlist supports global, organizational, and user-defined scopes, enabling trusted domains to bypass warnings. User overrides are stored locally and do not influence model parameters, preventing feedback poisoning and preserving privacy. This design demonstrates how human-in-the-loop correction can be integrated without compromising model integrity or offline operation.

Integrated Discussion and Limitations

Taken together, the results demonstrate that an offline, URL-only phishing detection system can achieve state-of-the-art performance while remaining lightweight, interpretable, and deployable in real browsing environments. Compared to prior approaches that rely on deep neural architectures, backend infrastructure, or external services, the proposed system offers superior practicality without sacrificing detection quality.

Several limitations remain. The evaluation is conducted on a static dataset and does not explicitly model long-term concept drift or adaptive adversarial behavior. Additionally, reliance on URL-level features may limit detection in rare cases involving highly benign-looking URLs. These limitations define clear directions for future work, including update mechanisms that preserve offline constraints and complementary signals that do not compromise privacy.

Nevertheless, by demonstrating that accuracy, low latency, interpretability, user agency, and real-world deployability can be achieved simultaneously within a single coherent design, this work advances the practical state of phishing detection and provides a strong foundation for privacy-preserving browser security systems.

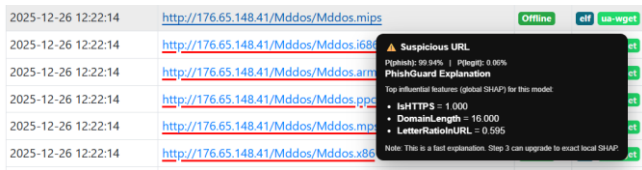


Fig. 2. Inline phishing warning generated by the proposed browser-based detection system, together with the corresponding SHAP-based feature explanation for the detected URL.

V. CONCLUSION

This paper presented a privacy-preserving, offline phishing URL detection system designed for real-time deployment in browser environments. By relying exclusively on intrinsic URL-based features and a carefully selected XGBoost classifier, the proposed approach achieves state-of-the-art detection performance while maintaining low inference latency and eliminating dependence on external services or network connectivity.

Experimental results on a large, contemporary phishing dataset demonstrate that the proposed system can simultaneously achieve high accuracy, low false-alarm rates, and low phishing miss rates, even under deployment-oriented constraints. Through threshold calibration, the same trained model can be adapted to different operational policies without retraining, enabling flexible trade-offs between security sensitivity and usability. The integration of SHAP-based explainability further enhances transparency by providing both global and local insights into model decisions, addressing a key limitation of many high-performing phishing detection systems.

Beyond offline evaluation, the system was validated through integration into a prototype browser extension that performs inline, non-intrusive phishing warnings with on-device explanations. This end-to-end deployment demonstrates that effective phishing defense does not require complex deep learning architectures, cloud-based inference, or extensive content inspection, and can instead be achieved through lightweight, interpretable models suitable for real-world use.

While the current evaluation is conducted on a static dataset and focuses on URL-level signals, the results establish a strong foundation for future extensions. Potential directions include adaptive update mechanisms that preserve offline constraints, robustness analysis under concept drift, and the incorporation of additional lightweight signals that do not compromise user privacy. Overall, this work advances the practical state of phishing detection and provides a viable blueprint for deployable, explainable, and privacy-conscious browser security systems.

ACKNOWLEDGMENT

The authors declare no conflict of interest. OpenAI's ChatGPT [16] was employed exclusively to improve language readability. The authors retain full responsibility for the scientific content of this work.

REFERENCES

- [1] D. Sisodia and D. S. Sisodia, "A transfer learning framework towards identifying behavioral changes of fraudulent publishers in pay-per-click model of online advertising for click fraud detection," *Expert Syst Appl*, vol. 232, p. 120922, Dec. 2023, doi: 10.1016/J.ESWA.2023.120922.
- [2] M. Sánchez-Paniagua, E. Fidalgo, E. Alegre, and R. Alaiz-Rodríguez, "Phishing websites detection using a novel multipurpose dataset and web technologies features," *Expert Syst Appl*, vol. 207, p. 118010, Nov. 2022, doi: 10.1016/J.ESWA.2022.118010.
- [3] D. Lain, E. Zurich, Y. Nakatsuka, K. Kostianen, G. Tsudik, and S. Capkun, "URL Inspection Tasks: Helping Users Detect Phishing Links in Emails," Feb. 2025, Accessed: Nov. 14, 2025. [Online]. Available: <https://arxiv.org/abs/2502.20234v1>
- [4] S. S. Roy, C. Torres, and S. Nilizadeh, "'Explain, Don't Just Warn!' -- A Real-Time Framework for Generating Phishing Warnings with Contextual Cues," May 2025, Accessed: Nov. 14, 2025. [Online]. Available: <https://arxiv.org/abs/2505.06836v2>
- [5] B. O. Ahubele and O. Nelson, "Phishing website detection using explainable artificial intelligence," *Dutse Journal of Pure and Applied Sciences*, vol. 11, no. 4b, pp. 114–122, Nov. 2025, doi: 10.4314/dujopas.v11i4b.11.
- [6] O. Research *et al.*, "Explainable Machine Learning for Phishing Site Detection: A High-Efficiency Approach Using Boosting Models and SHAP," *The Journal of Engineering*, vol. 2025, no. 1, p. e70110, Jan. 2025, doi: 10.1049/TJE2.70110.
- [7] Q. E. ul Haq, M. H. Faheem, and I. Ahmad, "Detecting Phishing URLs Based on a Deep Learning Approach to Prevent Cyber-Attacks," *Applied Sciences* 2024, Vol. 14, Page 10086, vol. 14, no. 22, p. 10086, Nov. 2024, doi: 10.3390/AP142210086.
- [8] S. Baskota, "Phishing URL Detection using Bi-LSTM," Apr. 2025, Accessed: Nov. 14, 2025. [Online]. Available: <https://arxiv.org/abs/2504.21049v1>
- [9] M. Almohaimeed, F. Albalwy, L. Algulaiti, and H. Althubayni, "Phishing URL Detection Using Deep Learning: A Resilient Approach to Mitigating Emerging Cybersecurity Threats," *Ingénierie des Systèmes d'Information*, vol. 30, no. 5, p. 1219, May 2025, doi: 10.18280/ISI.300510.
- [10] "(PDF) Client-Side Phishing Detection via Browser Extension: A Multi-Vector Approach Using ML and HTML Embeddings." Accessed: Dec. 26, 2025. [Online]. Available: https://www.researchgate.net/publication/397745688_Client-Side_Phishing_Detection_via_Browser_Extension_A_Multi-Vector_Approach_Using_ML_and_HTML_Embeddings
- [11] Z. J. Wang and D. H. Chau, "WebSHAP: Towards Explaining Any Machine Learning Models Anywhere," *ACM Web Conference 2023 - Companion of the World Wide Web Conference, WWW 2023*, pp. 262–266, Mar. 2023, doi: 10.1145/3543873.3587362.
- [12] A. Sharma, S. Rani, and M. Shabaz, "An optimized stacking-based TinyML model for attack detection in IoT networks," *PLoS One*, vol. 20, no. 8, p. e0329227, Aug. 2025, doi: 10.1371/JOURNAL.PONE.0329227.
- [13] "PhiUSIIL Phishing URL (Website) - UCI Machine Learning Repository." Accessed: Dec. 26, 2025. [Online]. Available: <https://archive.ics.uci.edu/dataset/967/phiusiil+phishing+url+dataset>
- [14] J. Chen, F. Zhao, Y. Sun, and Y. Yin, "Improved XGBoost model based on genetic algorithm," *International Journal of Computer Applications in Technology*, vol. 62, no. 3, pp. 240–245, 2020, doi: 10.1504/IJCAT.2020.106571.
- [15] D. M. W. Powers and Ailab, "EVALUATION: FROM PRECISION, RECALL AND F-MEASURE TO ROC, INFORMEDNESS, MARKEDNESS & CORRELATION".
- [16] OpenAI, "ChatGPT large language model," OpenAI, Dec. 2025. [Online]. Available: chat.openai.com